

```
In [1]: # The Iris Setosa
from IPython.display import Image
url = 'http://upload.wikimedia.org/wikipedia/commons/5/56/Kosaciec_szczecinkowaty_I
Image(url,width=300, height=300)
```

Out[1]:



```
In [2]: # The Iris Versicolor
from IPython.display import Image
url = 'http://upload.wikimedia.org/wikipedia/commons/4/41/Iris_versicolor_3.jpg'
Image(url,width=300, height=300)
```

Out[2]:



```
In [3]: # The Iris Virginica
from IPython.display import Image
url = 'http://upload.wikimedia.org/wikipedia/commons/9/9f/Iris_virginica.jpg'
Image(url,width=300, height=300)
```

Out[3]:



```
In [46]: import seaborn as sns
```

```
In [47]: iris = sns.load_dataset('iris')
```

```
In [48]: iris.keys()
```

```
Out[48]: Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',  
              'species'],  
              dtype='object')
```

```
In [49]: iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 5 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   sepal_length    150 non-null   float64  
1   sepal_width     150 non-null   float64  
2   petal_length    150 non-null   float64  
3   petal_width     150 non-null   float64  
4   species         150 non-null   object  
dtypes: float64(4), object(1)  
memory usage: 6.0+ KB
```

```
In [50]: iris.head()
```

```
Out[50]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [51]: iris.shape
```

Out[51]: (150, 5)

```
In [52]: iris.species.value_counts()
```

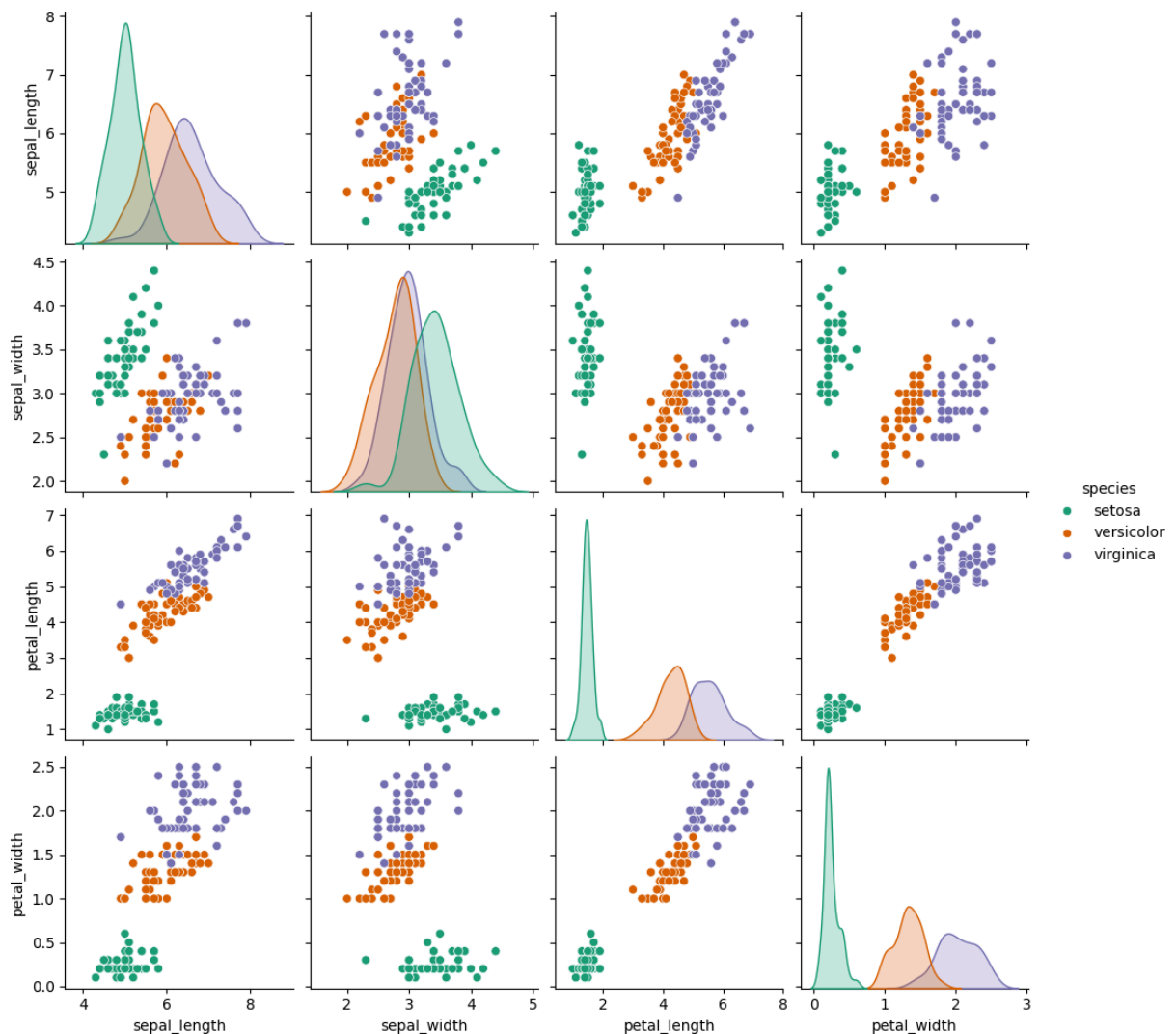
```
Out[52]: species
setosa      50
versicolor  50
virginica    50
Name: count, dtype: int64
```

Exploratory Data Analysis

```
In [53]: import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

```
In [54]: sns.pairplot(iris, hue='species', palette='Dark2')
```

Out[54]: <seaborn.axisgrid.PairGrid at 0x128f8a71b20>



Create a kde plot of sepal_length versus sepal width for setosa species of flower.

```
In [55]: setosa = iris[iris['species']=='setosa']
```

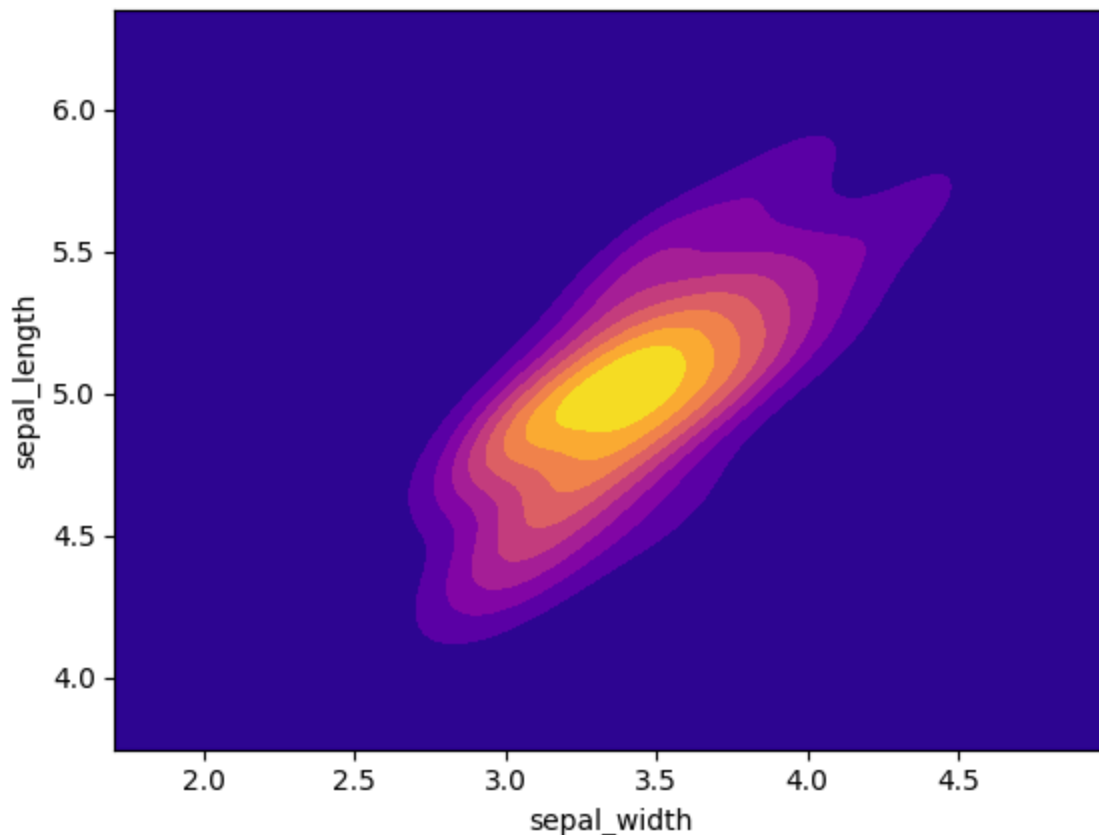
```
In [56]: setosa.head(2)
```

```
Out[56]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa

```
In [57]: sns.kdeplot(data=setosa,x= 'sepal_width',y='sepal_length',cmap='plasma',fill=True,t
```

```
Out[57]: <Axes: xlabel='sepal_width', ylabel='sepal_length'>
```



Encode the Target Variable

```
In [58]: from sklearn.preprocessing import LabelEncoder  
le = LabelEncoder()  
iris['species'] = le.fit_transform(iris['species']) # Convert to numeric labels
```

```
In [59]: iris['species'].value_counts()
```

```
Out[59]: species
0      50
1      50
2      50
Name: count, dtype: int64
```

Split the Data

```
In [60]: from sklearn.model_selection import train_test_split
X = iris.drop('species', axis=1)
y = iris['species']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

Train and Evaluate Machine Learning Models

Random Forest Classifier

```
In [61]: from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier()
rf_model.fit(X_train,y_train)
```

```
Out[61]: ▼ RandomForestClassifier ⓘ ?
RandomForestClassifier()
```

```
In [62]: y_pred_rf = rf_model.predict(X_test)
```

```
In [63]: from sklearn.metrics import accuracy_score,confusion_matrix,classification_report
```

```
In [64]: # Evaluate the model
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Classification Report:\n", classification_report(y_test, y_pred_rf))
print('Confutaion Matrics')
print(confusion_matrix(y_test,y_pred_rf))
```

Random Forest Accuracy: 0.9666666666666667

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.92	1.00	0.96	12
2	1.00	0.88	0.93	8
accuracy			0.97	30
macro avg	0.97	0.96	0.96	30
weighted avg	0.97	0.97	0.97	30

Confutaion Matrics

```
[[10  0  0]
 [ 0 12  0]
 [ 0  1  7]]
```

Support Vector Machine (SVM)

```
In [65]: from sklearn.svm import SVC
svm_model = SVC()
svm_model.fit(X_train,y_train)
```

Out[65]:

SVC

SVC()

```
In [66]: svm_pred = svm_model.predict(X_test)
```

```
In [67]: # Evaluate the model
print("Support Vector Machine Accuracy:", accuracy_score(y_test, svm_pred))
print("Classification Report:\n", classification_report(y_test, svm_pred))
print('Confution Matri')
print(confusion_matrix(y_test,svm_pred))
```

Support Vector Machine Accuracy: 0.9666666666666667

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.92	1.00	0.96	12
2	1.00	0.88	0.93	8
accuracy			0.97	30
macro avg	0.97	0.96	0.96	30
weighted avg	0.97	0.97	0.97	30

Confution Matri

```
[[10  0  0]
 [ 0 12  0]
 [ 0  1  7]]
```

Logistic Regression model**

```
In [68]: from sklearn.linear_model import LogisticRegression
lr_model = LogisticRegression(max_iter=200, random_state=42)
```

```
In [69]: lr_model.fit(X_train,y_train)
```

```
Out[69]: LogisticRegression
LogisticRegression(max_iter=200, random_state=42)
```

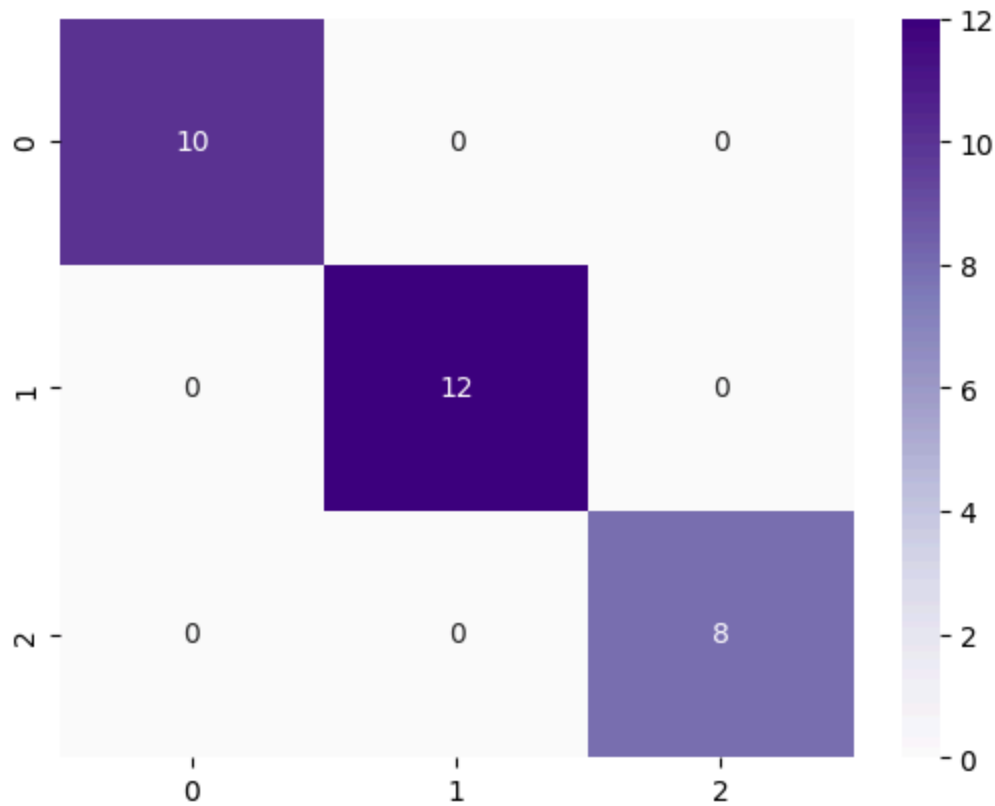
```
In [70]: pred_lr = lr_model.predict(X_test)
```

```
In [71]: # Evaluate the model
print("Support Vector Machine Accuracy:", accuracy_score(y_test, pred_lr))
print("Classification Report:\n", classification_report(y_test, pred_lr))
cm = confusion_matrix(y_test, pred_lr)
sns.heatmap(cm,annot=True,fmt='d',cmap='Purples')
plt.show()
```

Support Vector Machine Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	12
2	1.00	1.00	1.00	8
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



K-Nearest Neighbors (KNN)

```
In [72]: from sklearn.neighbors import KNeighborsClassifier  
KN_model = KNeighborsClassifier(n_neighbors=5)
```

```
In [73]: KN_model.fit(X_train,y_train)
```

```
Out[73]: KNeighborsClassifier ⓘ ?  
KNeighborsClassifier()
```

```
In [74]: KN_pred = KN_model.predict(X_test)
```

```
In [75]: # Evaluate the model  
print("K-Nearest Neighbors Accuracy:", accuracy_score(y_test, KN_pred ))  
print("Classification Report:\n", classification_report(y_test, KN_pred ))  
print('Confution Matri')  
print(confusion_matrix(y_test,KN_pred))
```


K-Nearest Neighbors Accuracy: 1.0

Classification Report:

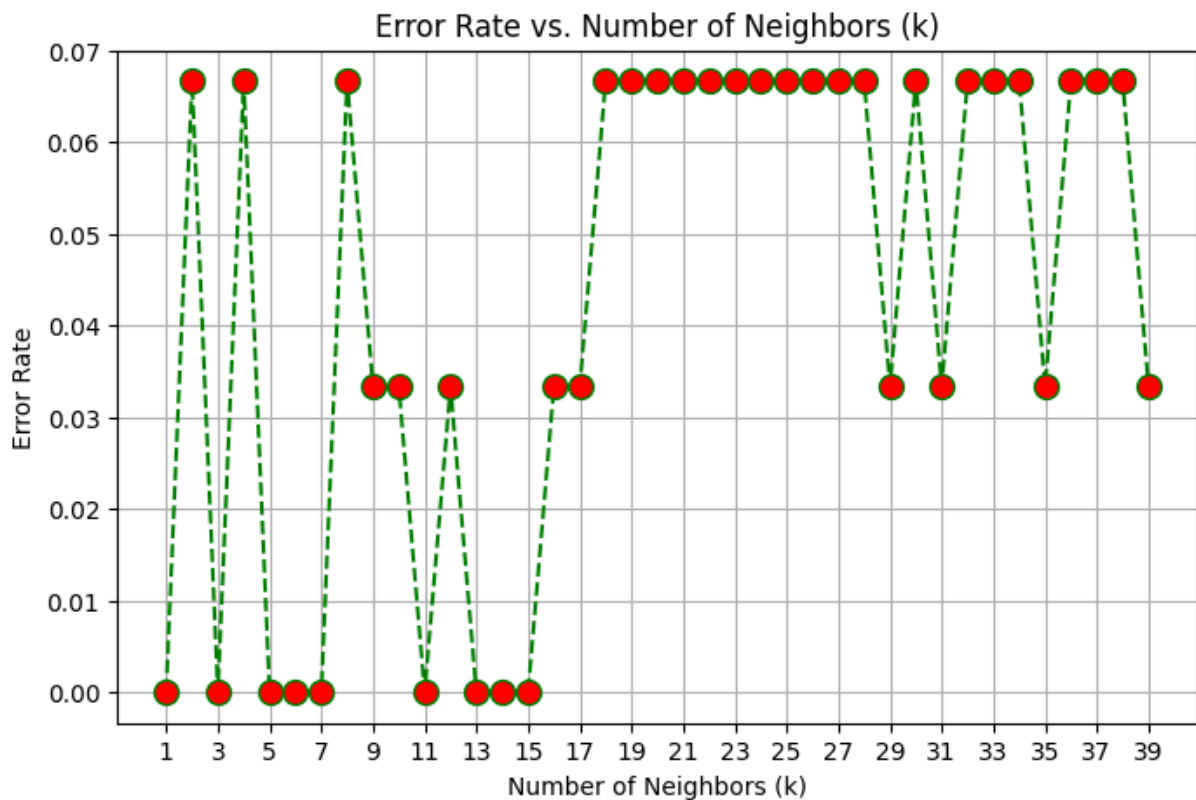
	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	12
2	1.00	1.00	1.00	8
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Confution Matri

```
[[10  0  0]
 [ 0 12  0]
 [ 0  0  8]]
```

```
In [76]: import numpy as np
error_rate=[]
for i in range(1,40):
    knn= KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train,y_train)
    pred_i=knn.predict(X_test)
    error_rate.append(np.mean(pred_i != y_test))
```

```
In [77]: # Plot the error rate for each value of n_neighbors
plt.figure(figsize=(8, 5))
plt.plot(range(1, 40), error_rate, marker='o', linestyle='--', color='g', markersize=10)
plt.title('Error Rate vs. Number of Neighbors (k)')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Error Rate')
plt.xticks(range(1, 40, 2)) # Show only odd numbers for readability
plt.grid()
plt.show()
```



```
In [78]: # Train the KNN model with the optimal number of neighbors
knn_optimal = KNeighborsClassifier(n_neighbors= 11)
knn_optimal.fit(X_train, y_train)

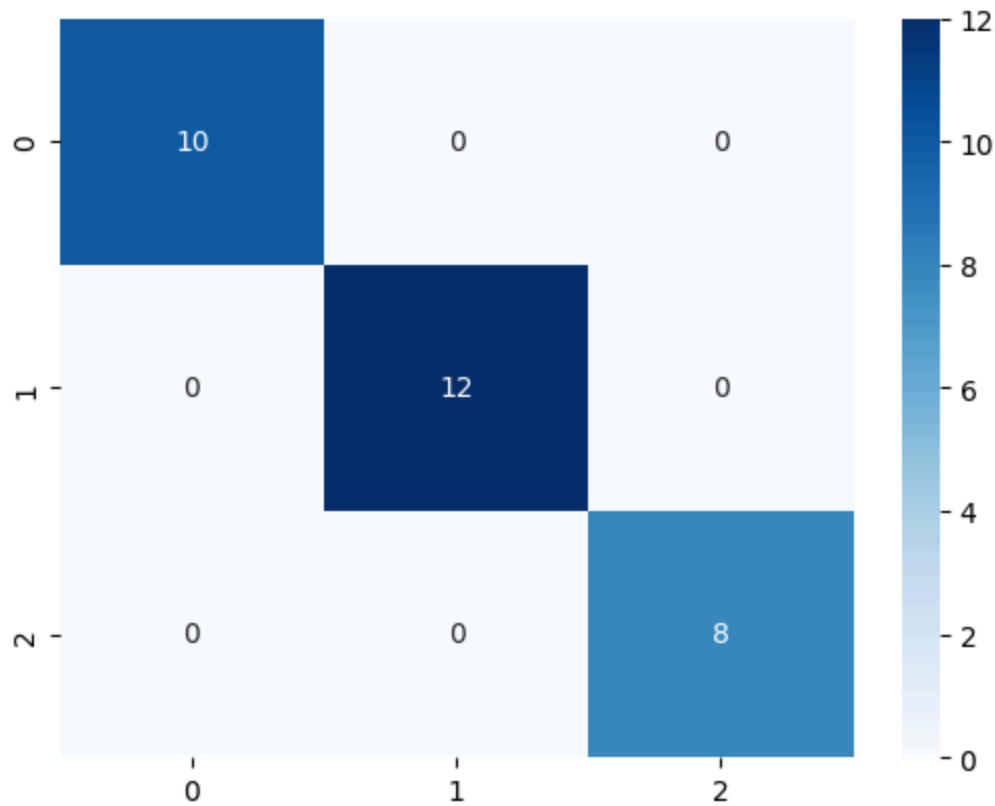
y_pred_optimal = knn_optimal.predict(X_test)
```

```
In [79]: # Evaluate the model
print("K-Nearest Neighbors optimal Accuracy:", accuracy_score(y_test, y_pred_optimal))
print("Classification Report:\n", classification_report(y_test, y_pred_optimal))
cm = confusion_matrix(y_test, y_pred_optimal)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.show()
```

K-Nearest Neighbors optimal Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	12
2	1.00	1.00	1.00	8
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



Hyperparameter Tuning**

```
In [80]: from sklearn.model_selection import GridSearchCV

# Define the parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
}

# Grid Search
grid_search = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=
grid_search.fit(X_train, y_train)
```

```
Out[80]: GridSearchCV
  ▸ best_estimator_: RandomForestClassifier
    ▸ RandomForestClassifier
```

```
In [81]: grid_search.best_params_
```

```
Out[81]: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 50}
```

```
In [82]: grid_search.best_estimator_
```

Out[82]:

```
RandomForestClassifier
RandomForestClassifier(n_estimators=50, random_state=42)
```

In [83]: `y_pred_grid = grid_search.predict(X_test)`

In [84]: `# Evaluate the model`
`print("Tuned Random Forest Accuracy:", accuracy_score(y_test, y_pred_grid ))`
`print("Classification Report:\n", classification_report(y_test, y_pred_grid ))`
`print('Confution Matri')`
`print(confusion_matrix(y_test,y_pred_grid))`

Tuned Random Forest Accuracy: 0.9666666666666667

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.92	1.00	0.96	12
2	1.00	0.88	0.93	8
accuracy			0.97	30
macro avg	0.97	0.96	0.96	30
weighted avg	0.97	0.97	0.97	30

Confution Matri

```
[[10  0  0]
 [ 0 12  0]
 [ 0  1  7]]
```

Best Model choice:

In [85]: `# Example of accuracy scores for different models`
`print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))`
`print('\n')`
`print("SVM Accuracy:", accuracy_score(y_test, svm_pred))`
`print('\n')`
`print("Logistic Regression Accuracy:", accuracy_score(y_test, pred_lr))`
`print('\n')`
`print("KNN Accuracy:", accuracy_score(y_test, KN_pred))`
`print('\n')`
`print("K-Nearest Neighbors optimal Accuracy:", accuracy_score(y_test, y_pred_optimal))`
`print('\n')`
`print("Tuned Random Forest Accuracy:", accuracy_score(y_test, y_pred_grid ))`

Random Forest Accuracy: 0.9666666666666667

SVM Accuracy: 0.9666666666666667

Logistic Regression Accuracy: 1.0

KNN Accuracy: 1.0

K-Nearest Neighbors optimal Accuracy: 1.0

Tuned Random Forest Accuracy: 0.9666666666666667

Best Model(s):

**Logistic Regression and KNN (including optimal KNN) achieve perfect accuracy (1.0).
However:

**Logistic Regression is generally simpler and faster, making it a good choice if the dataset is linearly separable or nearly so.

**KNN can work well but might be computationally expensive for large datasets because it needs to compute distances for all training points.

Logistic Regression Is the Best Model

**Reason: Logistic Regression is simple, interpretable, and achieved the highest accuracy. Given the Iris dataset's characteristics (small, linearly separable), Logistic Regression is the most appropriate and efficient choice.

**Secondary Option: KNN with default settings also achieved 100% accuracy, but Logistic Regression's interpretability and stability make it preferable.

Thank You

<https://www.linkedin.com/in/monirul-m08/> <https://github.com/MonirulIslam08>

**md08monirul@gmail.com